COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 Data Structures And Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/16/26	**DATE SUBMITTED:**	10/16/26
NAME:	Acala, Matt John C. Esteta, Jesse Matthew	**INSTRUCTOR:**	Engr. Jimlord M.Quejado

HOA 13.1 Parallel Algorithms and Multithreading

1. PROCEDURE OUTPUT

Table 13-1. Simple One-Threaded Example

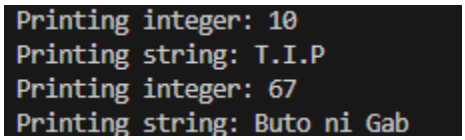
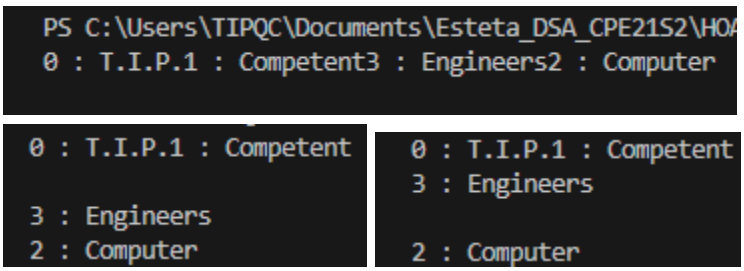
Screenshot	
Analysis	The main() function spawns a single worker thread t1 and passes the print function along with arguments 10 and "T.I.P.". Calling t1.join() blocks the main thread until t1 finishes printing both lines. Without t1.join(), main() could finish and destroy the std::thread object while t1 is still running, triggering std::terminate() at runtime.

Table 13-2. Multithreaded Example.

Screenshot	
Analysis	The program processed multiple threads at the same time where each run had different output which showed that multi-threading makes the process faster but it's not reliable as it gives different output each runtime.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.
Questions:
1. Write a definition of multithreading and its advantages/disadvantages.
Acala:
Multithreading is a programming and execution model where a single a single process spawns multiple

independent threads that run concurrently while sharing the same task, variables, etc…

Esteta:
MultiThreading allows one program to run different parts at the same time. Its advantage is that it makes the processing faster while its disadvantage is that it uses more resources and is incompatible to data that requires modifications.

2. Rationalize the use of multithreading by providing at least 3 use-cases.
Acala:
In graphical applications running DB, I/O file handling. Also in sorting algorithms and image filtering.

Esteta:
Multi Threading is used on Web browsers where texts and images are loaded separately, word processors: one thread handles the grammar / spelling checking and another for the typing. Lastly, media players one thread plays the video while another buffers the next part.
3. Differentiate between parallelism and concurrency.
Acala:
A concurrent program can run on a single-core processor while parallel requires multicore hardware to run independently while working simultaneously.

Esteta:

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

```
#include <iostream>
#include <thread>

int globalVar = 0;

void add(int value) {

    globalVar += value;

}

int main() {
    std::cout << "Display 1 (Before T1.join()): globalVar = " <<
globalVar << std::endl;

    std::thread T1(add, 10);

    T1.join();
```

```
    std::cout << "Display 2 (After T1.join()):  globalVar = " <<
globalVar << std::endl;

    std::thread T2(add, 20);

    T2.join();

    std::cout << "Display 3 (After T2.join()):  globalVar = " <<
globalVar << std::endl;

    std::thread T3(add, 30);

    T3.join();

    std::cout << "Display 4 (After T3.join()):  globalVar = " <<
globalVar << std::endl;

    return 0;

}
```

```
Display 1 (Before T1.join()): globalVar = 0
Display 2 (After T1.join()):  globalVar = 10
Display 3 (After T2.join()):  globalVar = 30
Display 4 (After T3.join()):  globalVar = 60
```

Acala: The main thread then spawns the first thread to add ten to the variable, immediately calling join to wait for the operation to complete before safely printing the updated value of ten. Next, a second thread is created to add twenty, and again, the main thread waits for it to join before correctly outputting the new total of thirty. Finally, the main thread spawns a third thread to add thirty, and calls join to ensure this final addition finishes completely before executing the last print statement.

Esteta: The program processed each thread first before joining them. In the first thread added 10 to the global var only after it was processed was thread 2 added to the global var. This process repeats for each thread until all threads are processed.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <utility>

int partition(std::vector<int> &arr, int low, int high)
{

    int pivot = arr[high];

    int i = low - 1;

    for (int j = low; j < high; j++)
    {

        if (arr[j] < pivot)
        {

            i++;

            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(std::vector<int> &arr, int low, int high)
{

    if (low < high)
    {

        int pi = partition(arr, low, high);
```

```
        quickSort(arr, low, pi - 1);

        quickSort(arr, pi + 1, high);
    }
}

void parallelQuickSort(std::vector<int> &arr, int low, int high)
{

    if (low < high)
    {

        int pi = partition(arr, low, high);

        std::thread leftThread(quickSort, std::ref(arr), low, pi - 1);

        std::thread rightThread(quickSort, std::ref(arr), pi + 1,
high);

        leftThread.join();

        rightThread.join();
    }
}

int main()
{

    std::vector<int> data = {42, 15, 8, 99, 23, 4, 16, 55, 12, 77};

    std::cout << "Unsorted array: ";

    for (int val : data)
    {

        std::cout << val << " ";
    }
```

```
    std::cout << std::endl;

    quickSort(data, 0, data.size() - 1);
    std::cout << "Sorted array using Quick Sort: ";

    for (int val : data)
    {

        std::cout << val << " ";
    }

    std::cout << std::endl;

    parallelQuickSort(data, 0, data.size() - 1);

    std::cout << "Sorted array using Multithreaded Quick Sort: ";

    for (int val : data)
    {

        std::cout << val << " ";
    }

    std::cout << std::endl;

    return 0;
}
```

```
Unsorted array: 42 15 8 99 23 4 16 55 12 77
Sorted array using Quick Sort: 4 8 12 15 16 23 42 55 77 99
Sorted array using Multithreaded Quick Sort: 4 8 12 15 16 23 42 55 77 99
```

Acala: We chose quick sort since it uses a another function called partition. It allows us to use 2 CPU cores to run simultaneously. Because 2 threads operate on strictly separate index bounds, no two threads read or write the same element at the same time, avoiding data races without needing mutex locks during the recursive sort.

Esteta: For this task we chose quick sort for multi threading as it divides the data to compare to low and high

which allowed us to do multi threading for processing the comparison for pivot on low and high at the same time. This allowed us to have a shorter run time as there is no repeating element since they are bound to different index partition.

Conclusion:

Acala: This activity covered how to implements concurrency and parallelism through std::thread. Threads execute functions asynchronously within a shared address space, and join() synchronizes thread completion with the calling thread so resources are not destroyed prematurely. While multithreading reduces execution time for independent tasks, sharing mutable variables across threads without synchronization introduces data races and non-deterministic output.

Esteta:
The activity covered how concurrency and parallelism is implemented using threads. Threading allows one to process one task then join it to the other threads after the previous thread was processed. Multi threading allows different threads to run at the same time which makes the process faster but makes it outputs unpredictable as the processing time for each data varies.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.